

NEW SECTION of `classification\_report` BEGINS HERE

Content

- * Problem with Accuracy metric
- * Precision
- * Recall
- * F1 score
- * Case study Bank Transaction
- * ROC & AUC
- * Cheat sheet for which metric to use when
- * Performance metric vs loss functions
- * Evaluating a classification model

****What are the issues with Accuracy?****

- * The accuracy metric doesn't work when we have imbalanced dataset
- * You may achieve an accuracy of 90% or more but is not a good score if 90 datapoints out of 100 belong to one class as you can achieve this score by always predicting the most common class value.
- * does not consider probabilities.

****QUIZ-1****

...

we have 500 samples such that there is 100 samples to class-1 and 400 samples to class-2

and the model predicts correct 5 samples as class-1 and remaining as class-2, what is the accuracy ?

1. 0.81
2. 1.05
3. 0.05
4. 0.95

...

ANSWER

1. 0.81

Explanation

$$\text{Accuracy} = \frac{(5+400)}{500} = 0.81$$

Discuss even though the model only predicts 5 samples of class-1

- it still manages to attain an accuracy of 0.81
- due to the imbalance in the dataset

Precision and Recall

Clearly, Accuracy wasn't a good indicator of performance for imbalanced data

What can be a good metric?

Let's understand two metrics Precision and Recall with a very simple example

Imagine you are fishing on a boat with a wide net in a lake.

- The lake has 100 fish and 100 turtles in it.

Since fishes catch a higher price in the market, your aim is to catch the fishes as much as possible in a single throw and avoid harming the turtles.

* Fish is your positive class and say turtle your negative.

* A fishing net can be considered as your ML model.

Let's start by understanding Precision first.

What does Precision mean intuitively?

-> Exactness

i.e. you want to be precise with your fishing skill so that you get as much fishes as possible

You throw a net and you'll catch some fishes and turtle.

Now, precision in terms of fishes will be out of all the catch, how many are fishes

Example

Say, you throw a normal fishing net in the water and you find 60 fish inside the net and 20 turtles too.

* Hence, your Precision is $\frac{60}{60+20}$

* These turtles will lower your Precision.

What will be the recall?

Recall can be thought of as out of all the fishes in lake, how many did you catch

- it is in terms of fishes only

Catch phrase: (Recall that they say, Lots of fish in the sea, how many did you catch?）**

How many total fishes do we have ?

-> 100

How many did we catch?

-> 60

Recall would be : $\frac{60}{100}$ i.e. 60%

i.e. we caught 60% of the fishes present in lake.

Second try

But your aim is to catch them in a single throw.

So you empty the net back in the water and this time you throw a bigger net in an attempt to catch all the fish.

* You manage to catch 80 fish ? but now you have also caught 80 turtles in the net! ?

Question: What will be the Precision?

-> We have total of 160 catches in the net

- Out of that 80 are fishes.

- Our precision is 50%

Question: What will be the Recall ?

Ans: We have 100 fish in lake and we caught 80.

- Hence, Recall is now 80% (as 80% of the fish was caught) but your Precision is 50% as just 50% of your catch is actually fish.

Third try

Atlast you use a third smaller net and when you throw it in the water

- you get just 20 fish but luckily no turtles.

What will be your precision and recall ?

You get a Recall of 20% but a Precision of 100% as everything you caught was fish.

****Precision****

Lets understand Precision seperately with some mathematical intuition

Question: Which situation do you think would be more critical?

1. Receiving a spam email in your inbox (think of FN)
2. Missing appreciation mail from manager (think of it as FP)

Obviously, appreciation mail from manager.

* Missing out on classifying spam as not spam is somewhat acceptable i.e False Negative

* However, classifying an important mail as spam can lead to major loss i.e. False Positive.

==> Reducing False Positive is more critical here

We want a metric that focuses on

- Reducing False Positive (not spam classified as spam), if FP ? ==> Metric ?
- Increasing True Positive (spam classified as spam), if TP ? ==> Metric ?

What metric can we use in this case?

We somehow need to ensure that out of all the emails which are predicted as spam, a high number of them are actually spam

OR

Out of all the emails predicted as spam, very low number of emails are not spam (low FP)

$$\frac{TP}{TP + FP} = \frac{\text{True Positive}}{\text{Predicted \space Positive}}$$

This metric is called Precision

Precision is the ratio of true positive to the predicted positives (sum of True positive and False Positive)

What does precision tell intuitively?

Intuitively, precision tell us out of all points predicted to be positive, how many are actually postives

Think about it, it basically tells, how precise our model is(with the predictions). Hence, the term precision.

Now, for our spam classification model,

- if False postives are high (important mail being classifed as spam)
- the denominator sum will go up
- overall precision score will drop.

Question: What do you think will be "range" of Precision?

=> Precision score will lie between 0 to 1.

We saw that Accuracy for our dumb model was 90%

Lets see what precision metrics tell us

How can we calculate Precision (for +ve class) using CM?

What will be the predictions for dumb model?

All non-spam (0/-ve)

Code

```
import pandas as pd
import numpy as np
# to get rid of warnings
import warnings
warnings.filterwarnings('ignore')
```

Code

```
# Run the below line to install a lower version if gdown throws any error
! pip install gdown==4.6.0
```

Output

```
Collecting gdown==4.6.0
Downloading gdown-4.6.0-py3-none-any.whl.metadata (4.4 kB)
Requirement already satisfied: filelock in /usr/local/lib/python3.12/dist-packages (from
gdown==4.6.0) (3.20.0)
Requirement already satisfied: requests[socks] in /usr/local/lib/python3.12/dist-packages
(from gdown==4.6.0) (2.32.4)
Requirement already satisfied: six in /usr/local/lib/python3.12/dist-packages (from
gdown==4.6.0) (1.17.0)
Requirement already satisfied: tqdm in /usr/local/lib/python3.12/dist-packages (from
gdown==4.6.0) (4.67.1)
Requirement already satisfied: beautifulsoup4 in /usr/local/lib/python3.12/dist-packages (from
gdown==4.6.0) (4.13.5)
Requirement already satisfied: soupsieve>1.2 in /usr/local/lib/python3.12/dist-packages (from
beautifulsoup4->gdown==4.6.0) (2.8)
Requirement already satisfied: typing-extensions>=4.0.0 in /usr/local/lib/python3.12/dist-
packages (from beautifulsoup4->gdown==4.6.0) (4.15.0)
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python3.12/dist-
packages (from requests[socks]->gdown==4.6.0) (3.4.4)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from
requests[socks]->gdown==4.6.0) (3.11)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages
(from requests[socks]->gdown==4.6.0) (2.5.0)
```

Code

```
!gdown 1_jZRFvpjElFBGy00-_4Cv68-n2cDbNUN
```

Output

```
Downloading...
From: https://drive.google.com/uc?id=1_jZRFvpjElFBGy00-_4Cv68-n2cDbNUn
To: /content/pred_data.csv
0% 0.00/247k [00:00<?, ?B/s]100% 247k/247k [00:00<00:00, 55.8MB/s]
```

Code

```
data = pd.read_csv('pred_data.csv')
data.head()
```

Output

```
y      proba
0  0.0  0.281035
1  0.0  0.465152
2  0.0  0.352793
3  0.0  0.157818
4  0.0  0.276648
```

Code

```
data.y.value_counts()
```

Output

```
y
0.0    10000
1.0      100
Name: count, dtype: int64
```

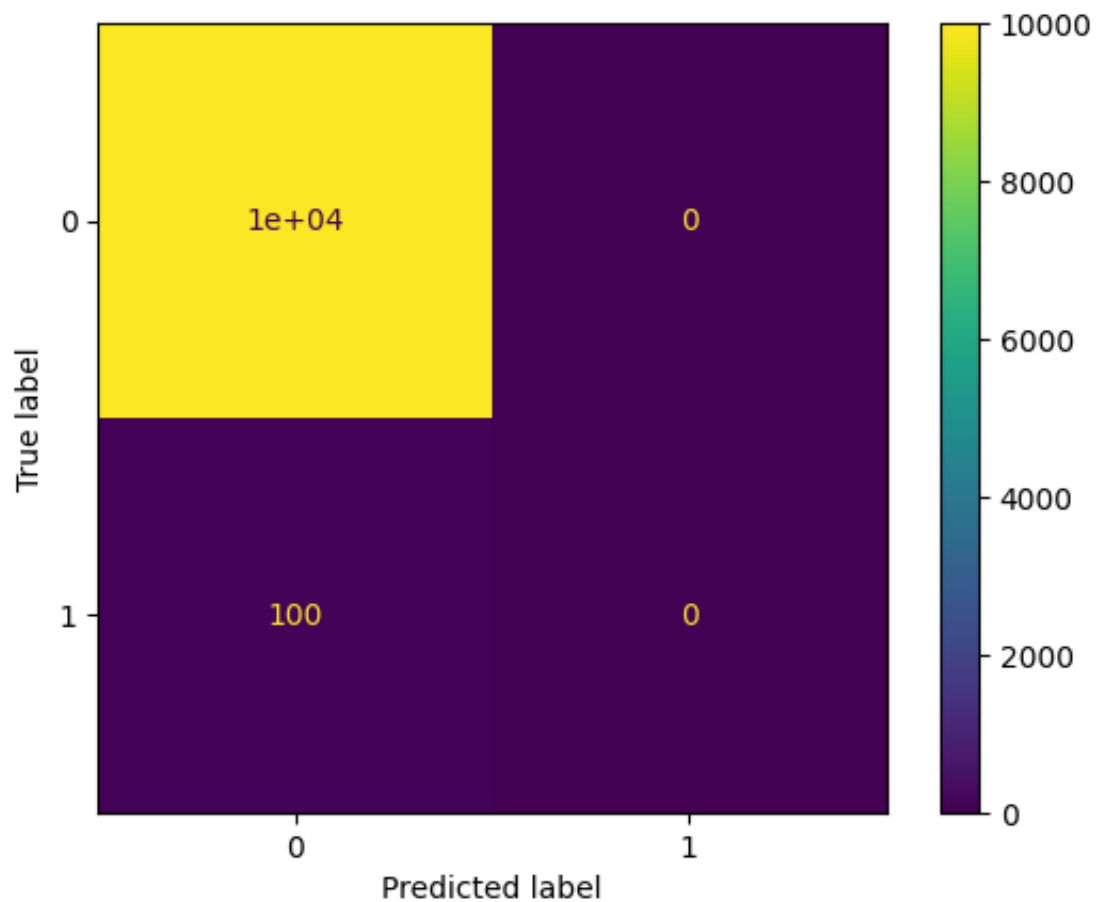
Code

```
# creating predictions based on the probabilities using the isgmoid logic
data['y_pred'] = data['proba'].apply(lambda y_score: 0.0 if y_score < 0.5 else 1.0)
```

Code

```
# Necessary imports
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
y_pred = np.zeros(len(data.y)) # predictions of a dumb model
conf_matrix = confusion_matrix(data.y, y_pred)
ConfusionMatrixDisplay(conf_matrix).plot();
```


Output Image



How do we get TP for +ve/1 class?

Diagonal element corresponding to class 1

Code

```
tp = conf_matrix[1, 1]  
tp
```

Output

```
np.int64(0)
```

How do we get Predicted Positive (for 1/+ve class)?

Column corresponding to class 1

Code

```
pred_pos = conf_matrix[:, 1].sum()  
pred_pos
```

Output

```
np.int64(0)
```

Usually, when we build a model, not all the predictions will 0

Code

```
pred_pos += pred_pos + 0.0000001 # add a small number in the denominator
```

Code

```
precision = tp / pred_pos  
precision
```

Output

```
np.float64(0.0)
```

Given y and y_pred, how can we calculate precision?

We can do it using sklearn library

Code

```
from sklearn.metrics import precision_score  
precision_score(data.y, y_pred)
```

Output

```
0.0
```

Looks like precision of the dumb model is 0

Lets check the precision for random model

Code

```
precision_score(data.y, data.y_pred)
```

Output

```
0.1870748299319728
```

Notice: Even random model > dumb model if it comes to making sure that we dont miss any important email

****Recall****

Imagine a scenario where we are building model to ****detect cancer****

The models task is to assist with the medical tests preliminary screening

Question: What will be +ve/-ve class in this case?

* Positive class: cancerous

* Negative class: non cancerous

What situation would be more critical?

1. Classify a healthy person as having cancer (False Positive)
2. Misclassify a person having cancer as healthy (False Negative)

Second situation can have dire consequences

* Classifying a healthy person as cancerous and carry out further testing is somewhat acceptable

* However, classifying person with cancer as healthy can be life death situation.

==> Reducing False Negatives is more critical here

We want a metric that focuses on

Reducing False Negatives (cancerous classified healthy), if FN ? ==> Metric ?

Increasing True Positive (cancerous classified as cancerous), if TP ? ==> Metric ?

What metric can we use for this case?

We want to make sure we don't miss out any cancerous patient

OR

We want to make sure that people having cancer are correctly classified

==> False Negative should be low, True Positives should be high

We want a metric that focuses on False negatives as well as True Positives

What metric do we use?

$$\frac{\text{True Positive}}{\text{True Positive} + \text{False Negative}} = \frac{\text{True Positive}}{\text{Actual Positives}}$$

This metric is called Recall

Recall is the ratio of True Positives to the actual positives (sum of TP + FN)

Think about it, it means, how good the model is recalling the actuals i.e. how many actual it predicted correctly (positive actuals)

What does Recall mean intuitively?

Intuitively, Recall tells us out of all the actually positive points, how many of them are predicted to be positive

Since False negative is in denominator

- when FN is low and TP is high

- recall will be high

Question: What will be the range of recall score?

Recall score will lie between 0 to 1.

Question: What do you think will be recall for dumb model for spam detection model?

Lets calculate it using CM

How can we calculate Recall (for +ve class) using CM?

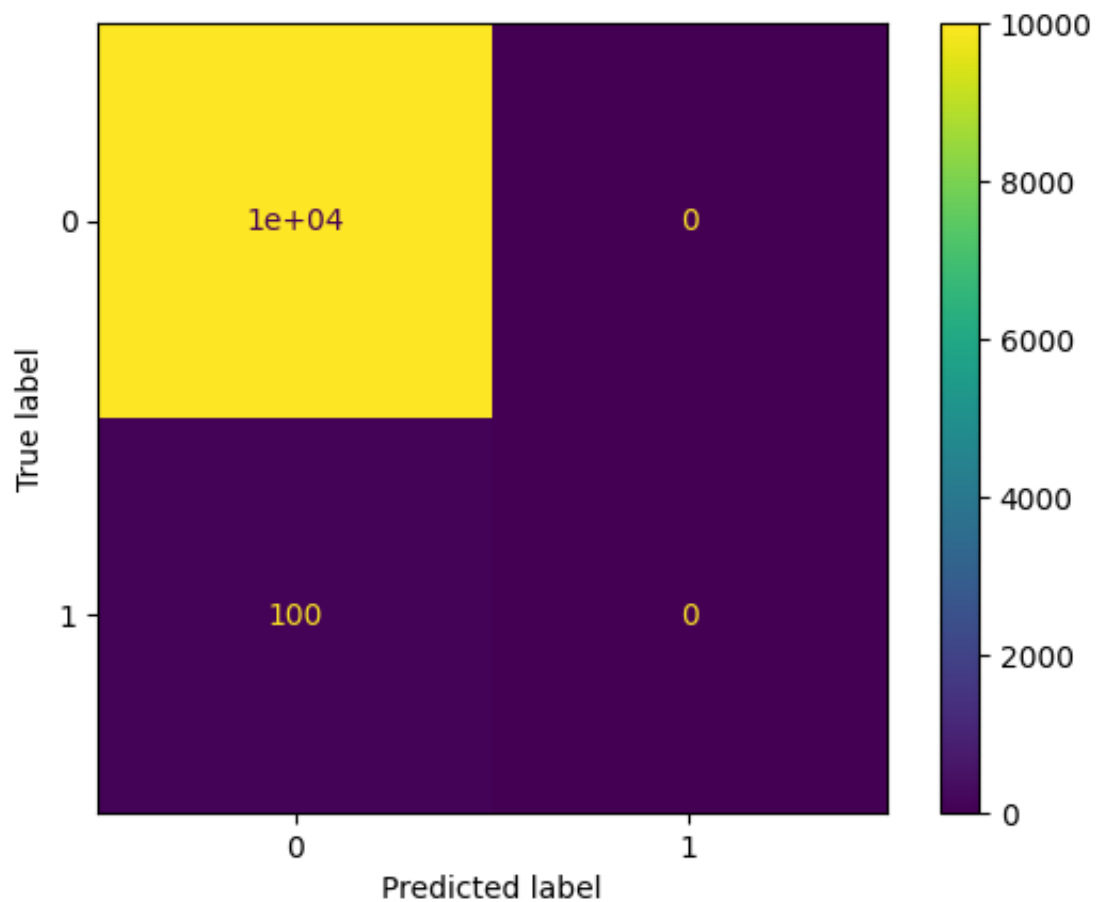
Code

```
y_pred = np.zeros(len(data.y)) # predictions of a dumb model
conf_matrix = confusion_matrix(data.y, y_pred)
ConfusionMatrixDisplay(conf_matrix).plot()
```

Output

```
<sklearn.metrics._plot.confusion_matrix.ConfusionMatrixDisplay at 0x7f5602e03c50>
```

Output Image



Code

```
tp = conf_matrix[1,1]
```

Question: How can we get actual positives from CM?

Code

```
actual_positives = conf_matrix[1,:].sum()
```

Code

```
recall = tp/actual_positives  
print(recall)
```

Output

```
0.0
```

Lets calculate it using sklearn as well

Code

```
from sklearn.metrics import recall_score
```

Code

```
recall_score(data.y, y_pred)
```

Output

```
0.0
```

What will be recall for random model ?

Code

```
recall_score(data.y, data.y_pred)
```

Output

0.55

Notice, for our spam detection model

* Precision was low (0.18), Recall is still high(er) (0.55)

* But remember, our focus here was to maximise the Precision

==> Both Precision and Recall values support our argument that dumb model isnt good.

****Summarizing Precision & Recall****

****Quiz-2****

...

Suppose you are building a model for movie recommendation

What would you prioritize more in this case?

A. High recall

B. High precision

...

ANSWER

A. High Precision

Explanation:

* It is okay to miss out on a good recommendation

* However, suggessting a bad recommendation will mean a user is spending less time on app. Hence, bad for business

Hence, Precision should be high

****QUIZ-3****

...

Suppose we are building a model to detect Fraudulent transaction

What should be prioritized more in this case?

A. High recall

B. High Precision

C. May be both

...

ANSWER

A. High Recall

EXPLANATION

* Tagging non fraudulent as fraudulent for further check is acceptable (False Positive).

* Missing out on fraudulent by marking them as non fraudulent will ruin the customer experience (False Negative)

In general (for most banks), maximising Recall is the goal

BUT

Working on customer experience is also important

Note: False Positives are handled by workflows like automated calls, messages to confirm or even human calls

****Lets look at some key-points towards summarising Precision and Recall****

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}} = \frac{\text{True Positive}}{\text{Predicted Positive}}$$

$$\text{Recall} = \frac{\text{True Positive}}{\text{True Positive} + \text{False Negative}}$$

Did you notice that there is no True Negative in both Precision and Recall?

* Precision deals with TP and FP

* Recall deals with TP and FN.

IMPORTANT

Both Precision and Recall are defined corresponding to the +ve class

BUT

If generalised to multi-class classification, we can calculate Precision and Recall w.r.t to class of interest

Are there hacks to remember Precision/Recall?

* When you cannot afford to have any false negatives (any misses in detecting the +ve), you prioritize recall.

* When you cannot afford to have any false positives (wrong +ve detections), you prioritize precision.

****QUIZ-4****

...

Consider two models A and B that each evaluate the same dataset. Which one of the following statements is true ideally?

1. If model A has better precision and better recall than model B, then model A is probably better.
2. If Model A has better precision than model B, then model A is better.
3. If model A has better recall than model B, then model A is better.
4. None of the these

...

Answer

5. If model A has better precision and better recall than model B, then model A is probably better.

****F1 score****

****Imagine you are working as Data Scientist for UpStart****

(company which provides loan based on education and employment instead of credit rating)

Task: Create loan prediction model

What do we want to predict?

Predict whether a person will be able to repay the loan or not.

What will be +ve/-ve class?

- * Positive class: Person will be able to repay loan
- * Negative class: Unable to repay the loan.

****What mindset will fintech company have for giving out loan?****

- * Loss to business if they give loan to people who are unable to repay it (False Positive)
- * Also a loss if they miss out on good people who will be able to repay (False Negative)

What metric should we use here?

Since we have to focus on both False Positive as well as False Negatives, we use both Precision and Recall

Comparing model based on precision recall

Suppose, you built 3 model and recorded their precision and recall score.

The scores are as follows:

Which model is better ?

* If we compare models based on precision, then model C is best

* If we compare models based on Recall, then model A is best.

Clearly, it is difficult to compare model based on two metrics.

How do we compare two model?

What if we combine precision and recall into a single metric?

How do combine these two metrics?

We want to create a metric such that

- when precision is high and recall is high ==> metric is high as well

=> Lets take simple average i.e. $(\text{Precision} + \text{Recall})/2$

Lets look at an example:

Issue with arth. mean of score ?

* When both Precision and Recall are 0.7

we get an score of 0.7

However, in second case

- precision rose to 0.9

- recall reduced to 0.5

- arth. mean remained the same.

Arithmetic Mean approach isnt penalizing the reduction in recall .

We want a metric which

- penalizes the drop in precision and recall (arithmetic mean is failing here)

- should be high when both precision and recall are high.

What if we use **Harmonic mean** ?

Notice that for Harmonic mean

- when both Precision and recall were 0.7, the score was 0.7

- However, in second case when

- precision rose to 0.9

- recall dropped to 0.5

- Harmonic mean dropped to 0.642

Harmonic mean is penalizing the drop in precision and recall.

This Harmonic mean is nothing but F1- score

$$F1\text{-score} = \frac{2}{\frac{1}{\text{Precision}} + \frac{1}{\text{Recall}}} = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

Hack to remember: We are using Harmonic mean as we are trying to find harmony between the two.

****QUIZ 5****

...

Why does the F-1 score use Harmonic Mean (HM) instead of Arithmetic Mean (AM) ?

1. HM penalizes models the most when even Precision and Recall are low.
2. AM penalizes models the most when even Precision and Recall are low.
3. HM penalizes models the most when even Precision and Recall are high.
4. AM penalizes models the most when even Precision and Recall are high.

...

ANSWER

1. HM penalizes models the most when even Precision and Recall are low.

Why not use Geometric Mean?

Lets try to understand it using a plot

Code

```
## Run the below line to install a lower version if gdown throws any error
#pip install gdown==4.6.0
#OR
#!wget --no-check-certificate
'https://docs.google.com/uc?export=download&id=1RspZN_Sam6zPVYjw2hInZzwhtiE8RXrp' -O
plot.html
```

Code

```
!gdown 1RspZN_Sam6zPVYjw2hInZzwhtiE8RXrp
```

Output

```
Downloading...  
From: https://drive.google.com/uc?id=1RspZN_Sam6zPVYjw2hInZzwhtiE8RXrp  
To: /content/plot.html  
100% 3.72M/3.72M [00:00<00:00, 19.3MB/s]
```

Code

```
import IPython  
IPython.display.HTML(filename='/content/plot.html')
```

Output

```
<IPython.core.display.HTML object>
```

Notice few things here

- Arth mean (Red) doesnt penalize much as precision or recall value decreases
- Harmonic mean is penalizing the reduction in precision and recall more than Geometric mean

How to calculate F1 score?

Question: What will be the f1 score for dumb model?

Ans: 0 As both precision and recall are 0.

Can we calculate it using sklearn as well ?

Code

```
from sklearn.metrics import f1_score
```

Code

```
f1_score(data.y, data.y_pred)
```

Output

```
0.27918781725888325
```

Notice that,

- as precision is low
- it is dragging down the f1 score.

Case study - Bank Transaction

Lets say you were given 10,000 transaction data

- out of which 300 are fraud transactions
- remaining 9700 are non-fraud

You designed a model to classify a transaction as fraud or not

lets say you confusion matrix like this

What will be the accuracy of this model ?

* $(TN + TP) / (TN + FP + FN + TP)$

* $9100 / 10000$

* $0.91 \Rightarrow 91\%$

what will be the recall for this model ?

* $(TP) / (FN + TP)$

* $100 / (100 + 200)$

* 0.333

what will be the precision for this model ?

* $(TP)/(FP+TP)$

* $100/(700+100)$

* 0.126

what will be the F1-score for this model ?

* $\frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$

* 0.182

what will be the FPR for this model ?

* $FPR = \frac{FP}{\text{Actual Negative}} = \frac{FP}{TN + FP}$

* $700/9700$

* 0.072

But suppose you manager say, that he has a dumbo model which will perform better than this above and model confusion matrix looks like this

what will be the accuracy of this model ?

* 97%

Is this a good model ?

- lets check precision and recall

what will be the recall for this model ?

* $(TP)/(FN+TP)$

* 0

what will be the precision for this model ?

* $(TP)/(FP+TP)$

* 0

what will be the F1-score for this model ?

* again 0

So we can clearly see, even though accuracy was greater than previous model but precision, recall, F1-score turned out to be 0

NEW SECTION of `classification\_report` BEGINS HERE

An easier way to get all these things like , , and at one place is by using classification report from sklearn.

Lets look into its implementation

Code

```
# importing classification report
from sklearn.metrics import classification_report
```

Code

```
# printing the classification report
print(classification_report(data.y, data.y_pred))
```

Output

```
precision    recall  f1-score   support

0.0         1.00      0.98      0.99     10000
1.0         0.19      0.55      0.28       100
accuracy               0.97     10100
macro avg           0.59      0.76      0.63     10100
weighted avg           0.99      0.97      0.98     10100
```

What all you can see here:

1. The overall accuracy of the model that we have created.
2. Support tells you about the count of points associated with the classes respectively.
3. We can see that the Precision, Recall and f1-score associated with each class are mentioned separately in front of each class.
4. Macro average is calculated as the mean of the values of a specific metric along all the classes.

macro avg for precision is calculated as $(0.19+1.00)/2 = 0.59$

1. Weighted average is calculated with the help of support values associated with each of the classes.

weighted avg for precision is calculated as $((1.00 \times 10,000) + (0.19 \times 100)) / 10100$.

Classification report makes it super easy for the tasks of multiclass classification as it provides great readability and interpretation for the various metrics associated with all the classes that we are predicting.

NEW SECTION ENDS HERE!

ROC

In above case study, we calculate FPR

What does this FPR=7.2% means ? is 7.2% a good or bad number ?

* 7.2% of non-fraud transaction were classified incorrectly

* So we want this percentage as low as possible

Now lets understand ROC and AUC with the help of an example

Lets say we have this data of 9 transactions where we are given

- transaction id
- True label where 0 means non-fraud and 1 being fraud
- Fraudulent probability
- and suppose we have threshold of 0.5 which means if probability is greater than 0.5 then it is fraud (1) else non-fraud (0)
- also notice we are misclassifying some transactions (2nd and 4th) as 1 marked in red

Note: transaction 6th is also misclassified but we will deal with it later

Now suppose we want to make all 1s label in correct category, but how ?

what if we change the threshold to 0.1

* now you can see all the 1s label are falling in correct category

What if we change threshold to 1 ?

- every transaction will be classified as 0

But should we do this ?

- No

Look at the confusion matrix for the above case

- we classified 5 fraud transactions as non-fraud

For threshold 1.0, What will be the value of FPR ?

- $0/(0+5)$

- 0

Similarly, TPR and F1-score will be 0

Now let's do the same calculations but with threshold 0.9

- Only last row will be affected

Similarly this will be the confusion matrix and values of FPR, TPR, F1-score

Let's look at this plot

- we are plotting TPR vs FPR when threshold is 0.9

Now let's plot this for other thresholds too

It will look like this]

Note : we first of all sort all transactions according to Fraudulent prob in ascending order

Note: Ideally we want our TPR to be 1 always but that will not be possible, curve will shift as we change threshold

****Question: What will be the best score for TPR?****

Ans: 1 ∴ when $TP = P$ i.e. No False Negatives $TPR = TP/P$

****Question: What will be the best FPR?****

Ans: 0 (when there are no false positives) $FPR = FP/N$

A model with $TPR = 1$ and $FPR = 0$ will be best model.

****Why is curve starting at (0,0)?****

Since we are classifying all the points as 0 except 1 point for first threshold (0.95), the TPR as well as FPR will be 0

****Why is right end of curve at (1,1) ?****

For n-th threshold, we are classifying every point as class 1.

Since all the points are being predicted as positive point

- Hence, TPR will 1 as all actual positive are predicted as positive
- Also, FPR will be 1 as all negative points are classified as positive as well.

****AUC****

Can we compute area under this graph ?

- * It is nothing but area under rectangle
- * If we add all these rectangles areas , it is nothing but AUC
- * Area here is $0.24 + 0.16 + 0.40 = 0.80$

Note: In ideal case area should be 1.0

Note: Here it was the dummy dataset so we got perfect rectangles but in real world dataset they will not be perfect rectangles

Question: What will be the AU-ROC for best model ?

The curve for best model will pass through best model point (0,1) and itll have an AUC of 1.

What will be the AUC for worst model?

The curve for worst model will pass through (1,0) and itll have an AUC of 0.

Lets say we have two models

- Model A (blue curve)
- Model B (cyan curve)

****Which model is better? A or B ?****

As the curve move away from (1,0) and towards (0,1), the model gets better

- As curve move towards (0,1), AUC increases.

AUC of Model B > Model A. Hence Model B is better

****Fundamental difference between previous metrics and AUC****

When we calculate Precision, Recall or F1 score

- We calculate it for a certain threshold on y_i
- This threshold is 0.5, by default

On the otherhand, AU-ROC

- we are calculating it using all possible thresholds

When we are comparing Model A and Model B, we are comparing it across all the thresholds and Model B is better than Model A across all the thresholds

****Properties of AU-ROC****

1. It does not work well for the highly imbalanced data set

It works for slightly imbalanced data but not for highly imbalanced data

2. AU-ROC does not depend on the actual values of the y_i but only depends on the order of them.

Lets say we have two model M1 and M2

- actual y labels are : [1, 1, 0, 1, 1]
- predicted probabilities for M1 are: [0.95, 0.92, 0.80, 0.76, 0.71]
- predicted probabilities for M2 are: [0.2, 0.1, 0.08, 0.06, 0.01]

Since, we will be using each probability value as threshold and calculating AUC

- the ordering of threshold isnt changing and hence TPR and FPR will be same.

1. Random Model has AU-ROC of 0.5
2. When models AOC is less than 0.5, the simple fix for the model is to invert your predictions. i.e After inverting you will get area of 1-(actual area value)

Code walkthrough

Code

```
from sklearn.metrics import roc_curve, roc_auc_score
from matplotlib import pyplot as plt
```

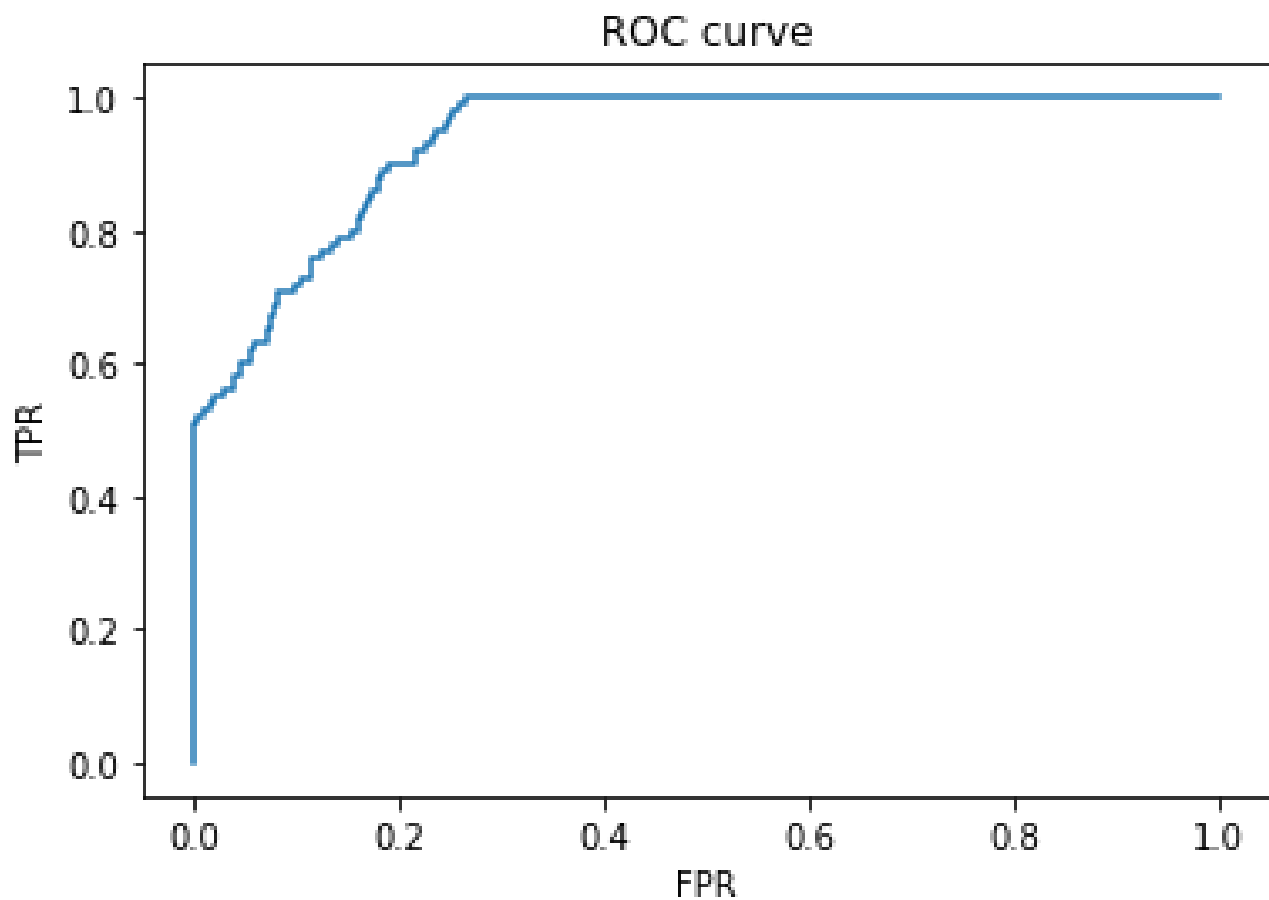
Code

```
fpr, tpr, thr = roc_curve(data.y, data.proba)
```

Code

```
plt.plot(fpr, tpr)
plt.title('ROC curve')
plt.xlabel('FPR')
plt.ylabel('TPR')
plt.show()
```

Output Image



Code

```
roc_auc_score(data.y, data.proba)
```

Output

```
0.9377570000000001
```

****QUIZ 6****

``

For the ROC curve of True-positive rates vs False-positive rates, which of the following is true?

1. The curve is always concave
2. The curve is never concave
3. The curve may or may not be concave

...

Answer

3. The curve may or may not be concave

Explanation

An ROC curve can be concave, convex, or a mix of those on different segments,

- The x-axis of the ROC curve is false positive rate, the y-axis is true-positive rate which is ideally a concave curve.

* Non-concave/Convex ROC curves would mean your TPR + FPR are low.

****Issue with AU-ROC****

AU- ROC doesnt work well on extremely imbalanced data.

However, we saw that Precision, recall and F1 score work well on highly imbalanced data.

So, what if we make curve using Precision Recall ? i.e. Precision Recall Curve

Cheat sheat for which metric to use when

* If we want probabilities of classes: Log loss

* If classes are balanced: Accuracy

* IF classes are imbalanced:

and if we are more concerned about False positive and true positive, then we use precision.If we are more concerned about False Negatives and True positive then we use recall.F1 score is a balance between precision and recall.If our concern is both classes (true negative and true positive) then we use ROC_AUC

* If severe imbalance : PR AUC

Performance Metric vs Loss functions

Q. How are performance metric different than Loss functions?

Loss functions show a measure of model performance. Theyre used to train a machine learning model (using some kind of optimization like Gradient Descent), and theyre usually differentiable in the models parameters.

Metrics are used to monitor and measure the performance of a model (during training and testing), and dont need to be differentiable.

However, if for some tasks the performance metric is differentiable, it can also be used as a loss function (perhaps with some regularizations added to it), such as MSE.

Evaluating a classification model

Code

```
## Run the below line to install a lower version if gdown throws any error
#pip install gdown==4.6.0
#OR
#!wget --no-check-certificate
'https://docs.google.com/uc?export=download&id=105fQvg0TsjH5uRlIHNxt-WByTg4SRfwI' -O
pima-indians-diabetes (1).csv
```

Code

```
!gdown 105fQvg0TsjH5uRlIHNxt-WByTg4SRfwI
```

Output

```
Downloading...
From: https://drive.google.com/uc?id=105fQvg0TsjH5uRlIHNxt-WByTg4SRfwI
To: /content/pima-indians-diabetes (1).csv
0% 0.00/23.8k [00:00<?, ?B/s]100% 23.8k/23.8k [00:00<00:00, 25.5MB/s]
```

Code

```
# read the data into a Pandas DataFrame
import pandas as pd
col_names = ['pregnant', 'glucose', 'bp', 'skin', 'insulin', 'bmi', 'pedigree', 'age',
'label']
```


Code

```
# print the first 5 rows of data
pima.head()
```

Output

	pregnant	glucose	bp	skin	insulin	bmi	pedigree	age	label
0	1	85	66	29	0	26.6	0.351	31	0
1	8	183	64	0	0	23.3	0.672	32	1
2	1	89	66	23	94	28.1	0.167	21	0
3	0	137	40	35	168	43.1	2.288	33	1
4	5	116	74	0	0	25.6	0.201	30	0

Code

```
# define X and y
feature_cols = ['pregnant', 'glucose', 'bp', 'skin', 'insulin', 'bmi', 'age']
X = pima[feature_cols]
y = pima.label
```

Code

```
# split X and y into training and testing sets
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, random_state=1)
```

Code

```
# train a logistic regression model on the training set
from sklearn.linear_model import LogisticRegression
logreg = LogisticRegression()
logreg.fit(X_train, y_train)
```

Output

```
LogisticRegression()
```

Code

```
X_test.info()
```

Output

```
<class 'pandas.core.frame.DataFrame'>
Int64Index: 192 entries, 285 to 629
Data columns (total 7 columns):
#   Column      Non-Null Count  Dtype
---  -
0   pregnant    192 non-null    int64
1   glucose     192 non-null    int64
2   bp          192 non-null    int64
3   skin        192 non-null    int64
4   insulin     192 non-null    int64
5   bmi         192 non-null    float64
6   age         192 non-null    int64
dtypes: float64(1), int64(6)
memory usage: 12.0 KB
```

Code

```
# make class predictions for the testing set  
y_pred_class = logreg.predict(X_test)
```

Classification accuracy: percentage of correct predictions

Code

```
# calculate accuracy  
from sklearn import metrics  
print(metrics.accuracy_score(y_test, y_pred_class))
```

Output

```
0.7395833333333334
```

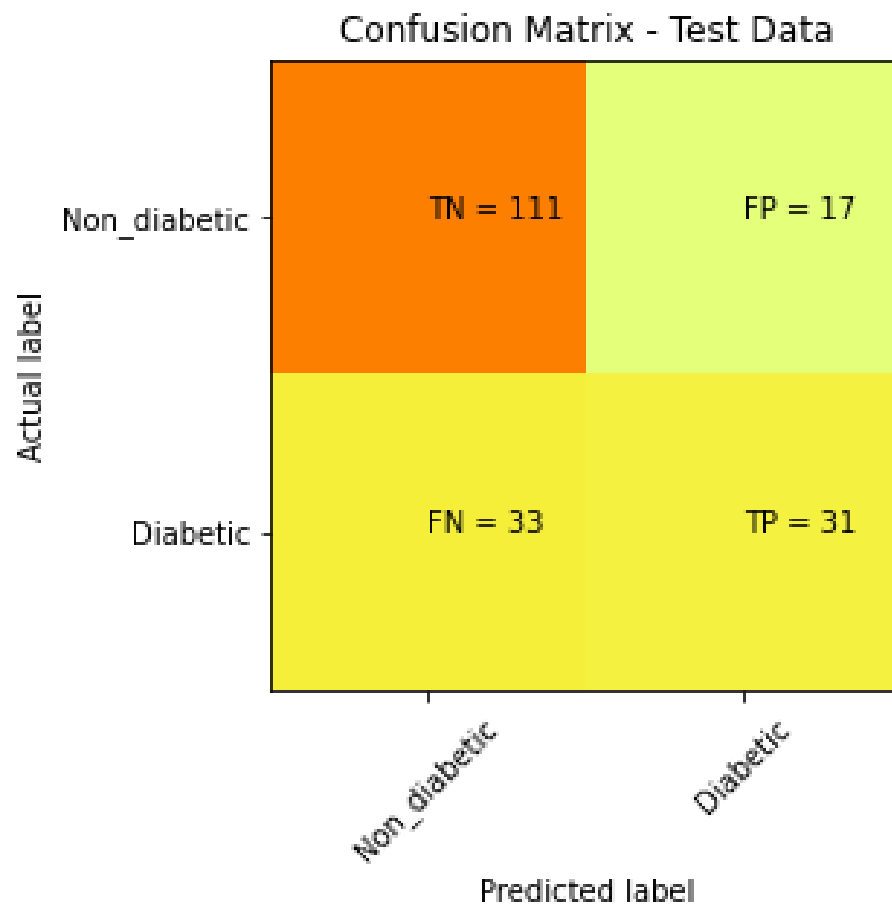
Confusion matrix

Table that describes the performance of a classification model

Code

```
import matplotlib.pyplot as plt
import numpy as np
cm = metrics.confusion_matrix(y_test, y_pred_class)
plt.clf()
plt.imshow(cm, interpolation='nearest', cmap=plt.cm.Wistia)
classNames = ['Non_diabetic', 'Diabetic']
plt.title('Confusion Matrix - Test Data')
plt.ylabel('Actual label')
plt.xlabel('Predicted label')
tick_marks = np.arange(len(classNames))
plt.xticks(tick_marks, classNames, rotation=45)
plt.yticks(tick_marks, classNames)
s = [['TN', 'FP'], ['FN', 'TP']]
for i in range(2):
    for j in range(2):
        plt.text(j, i, str(s[i][j])+" = "+str(cm[i][j]))
plt.show()
```

Output Image



Code

```
TP = 31
TN = 111
FP = 17
FN = 33
```

Code

```
/*TP = 40
TN = 110
FP = 13
```

Metrics computed from a confusion matrix

Classification Accuracy: Overall, how often is the classifier correct?

Code

```
print((TP + TN) / float(TP + TN + FP + FN))  
print(metrics.accuracy_score(y_test, y_pred_class))
```

Output

```
0.7395833333333334  
0.7395833333333334
```

Classification Error: Overall, how often is the classifier incorrect?

* Also known as Misclassification Rate

Code

```
print((FP + FN) / float(TP + TN + FP + FN))  
print(1 - metrics.accuracy_score(y_test, y_pred_class))
```

Output

```
0.2604166666666667  
0.2604166666666663
```

Sensitivity: When the actual value is positive, how often is the prediction correct?

* How sensitive is the classifier to detecting positive instances?

* Also known as True Positive Rate or Recall

Code

```
print(TP / float(TP + FN))  
print(metrics.recall_score(y_test, y_pred_class))
```

Output

```
0.484375  
0.484375
```

Specificity: When the actual value is negative, how often is the prediction correct?

* How specific (or selective) is the classifier in predicting positive instances?

Code

```
print(TN / float(TN + FP))
```

Output

```
0.8671875
```

False Positive Rate: When the actual value is negative, how often is the prediction incorrect?

Code

```
print(FP / float(TN + FP))
```

Output

```
0.1328125
```

Precision: When a positive value is predicted, how often is the prediction correct?

* How precise is the classifier when predicting positive instances?

Code

```
print(TP / float(TP + FP))  
print(metrics.precision_score(y_test, y_pred_class))
```

Output

```
0.6458333333333334  
0.6458333333333334
```

Adjusting the classification threshold

Code

```
# print the first 10 predicted class with default threshold of .5  
logreg.predict(X_test)[0:10]
```

Output

```
array([1, 0, 0, 0, 0, 0, 0, 0, 0, 0])
```


Code

```
# print the first 10 predicted probabilities of class membership
logreg.predict_proba(X_test)[0:10, :]
```

Output

```
array([[0.34824755, 0.65175245],
       [0.93297986, 0.06702014],
       [0.5370798 , 0.4629202 ],
       [0.94019163, 0.05980837],
       [0.8610544 , 0.1389456 ],
       [0.54081609, 0.45918391],
       [0.95008754, 0.04991246],
       [0.71692439, 0.28307561],
       [0.6956714 , 0.3043286 ],
       [0.99188101, 0.00811899]])
```

Code

```
# print the first 10 predicted probabilities for class 1 (diabetics)
logreg.predict_proba(X_test)[0:10, 1]
```

Output

```
array([0.65175245, 0.06702014, 0.4629202 , 0.05980837, 0.1389456 ,
       0.45918391, 0.04991246, 0.28307561, 0.3043286 , 0.00811899])
```

Code

```
# store the predicted probabilities for diabetic class for all records...
y_pred_prob = logreg.predict_proba(X_test)[: , 1]
```

Reduce the threshold from .5 to .3 to predict the diabetics class. This will make the model sensitive to diabetic class

Code

```
# predict diabetes if the predicted probability is greater than 0.3
from sklearn.preprocessing import binarize
y_pred_class = binarize([y_pred_prob], threshold =0.3)[0] # deciding the class of the 1st 10
records based on new threshold
```

Code

```
# print the first 10 predicted probabilities
y_pred_prob[0:10]
```

Output

```
array([0.65175245, 0.06702014, 0.4629202 , 0.05980837, 0.1389456 ,
       0.45918391, 0.04991246, 0.28307561, 0.3043286 , 0.00811899])
```

Code

```
# print the first 10 predicted classes with the lower threshold. Note the change in class...
# with threshold of .5 (default) , the first data point would belong to 0 class i.e.
non-diabetic
```

Output

```
array([1., 0., 1., 0., 0., 1., 0., 0., 1., 0.]
```

Code

```
# previous confusion matrix (default threshold of 0.5)  
print(metrics.confusion_matrix(y_test, y_pred_class))
```

Output

```
[[89 39]  
 [18 46]]
```

Code

```
# sensitivity has increased (used to be 0.24)  
print(46 / float(46 + 16))
```

Output

```
0.7419354838709677
```

Code

```
# specificity has decreased (used to be 0.91)  
print(80 / float(80 + 50))
```

Output

```
0.6153846153846154
```

Observations:

* Default threshold of .5 is not sensitive towards diabetic class. Lowering the threshold increases the sensitivity to

* diabetic class

ROC Curves and Area Under the Curve (AUC)

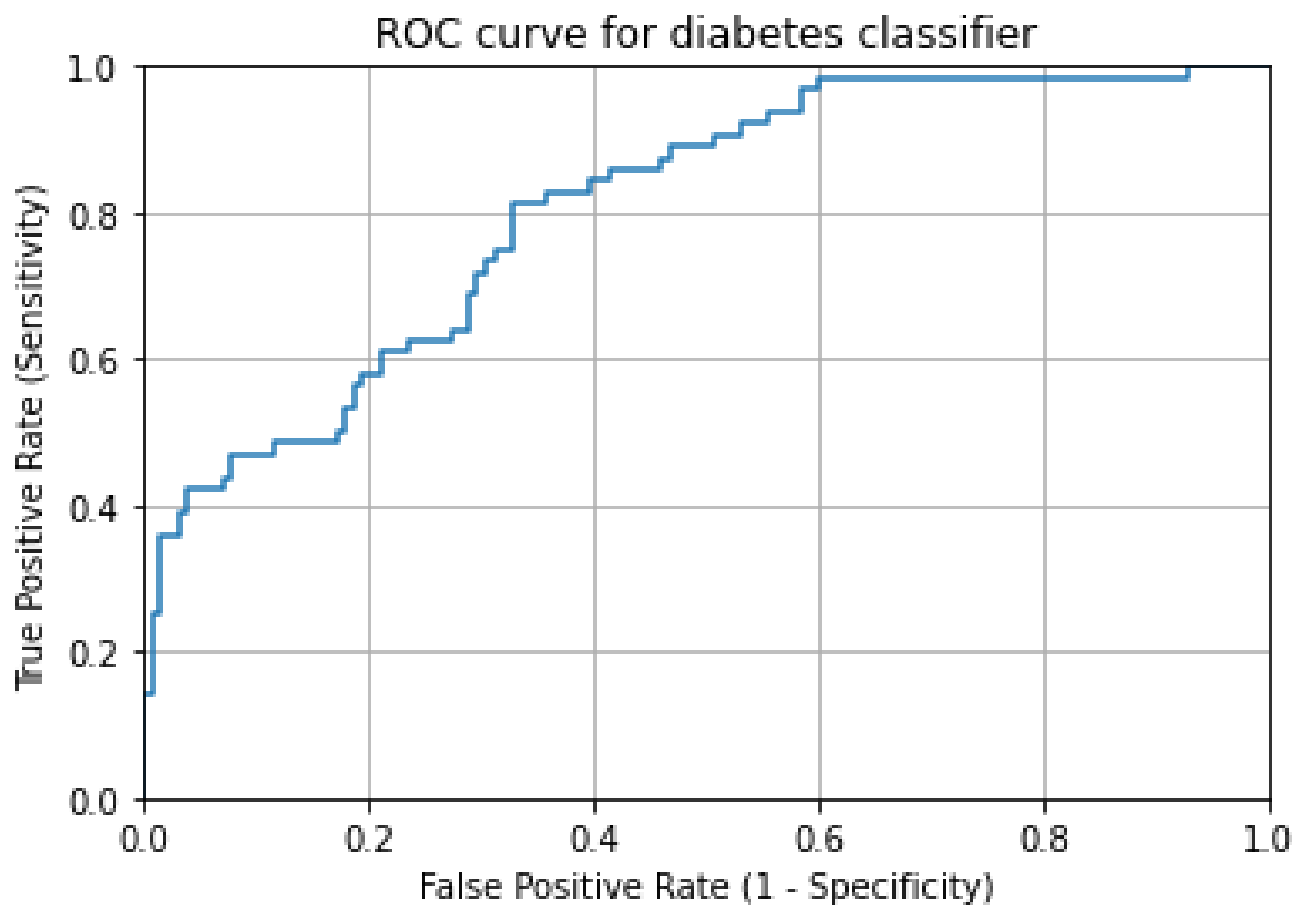
Question: Wouldnt it be nice if we could see how sensitivity and specificity are affected by various thresholds, without actually changing the threshold?

Answer: Plot the ROC curve!

Code

```
# IMPORTANT: first argument is true values, second argument is predicted probabilities
import matplotlib.pyplot as plt
fpr, tpr, thresholds = metrics.roc_curve(y_test, y_pred_prob)
plt.plot(fpr, tpr)
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.0])
plt.title('ROC curve for diabetes classifier')
plt.xlabel('False Positive Rate (1 - Specificity)')
plt.ylabel('True Positive Rate (Sensitivity)')
plt.grid(True)
```

Output Image



* ROC curve can help you to choose a threshold that balances sensitivity and specificity in a way that makes sense for your particular context

* You cant actually see the thresholds used to generate the curve on the ROC curve itself

Code

```
# define a function that accepts a threshold and prints sensitivity and specificity
def evaluate_threshold(threshold):
    print('Sensitivity:', tpr[thresholds > threshold][-1])
    print('Specificity:', 1 - fpr[thresholds > threshold][-1])
```

Code

```
evaluate_threshold(0.5)
```

Output

```
Sensitivity: 0.484375  
Specificity: 0.8828125
```

Code

```
evaluate_threshold(0.3)
```

Output

```
Sensitivity: 0.71875  
Specificity: 0.6953125
```

AUC is the percentage of the ROC plot that is underneath the curve:

Code

```
# IMPORTANT: first argument is true values, second argument is predicted probabilities  
print(metrics.roc_auc_score(y_test, y_pred_prob))
```

Output

```
0.80517578125
```

* AUC is useful as a single number summary of classifier performance.

* If you randomly chose one positive and one negative observation, AUC represents the likelihood that your classifier will assign a higher predicted probability to the positive observation.

* AUC is useful even when there is high class imbalance (unlike classification accuracy).

Code

```
# calculate cross-validated AUC
from sklearn.model_selection import cross_val_score
cross_val_score(logreg, X, y, cv=10, scoring='roc_auc').mean()
```

Error Output

```
/usr/local/lib/python3.7/dist-packages/sklearn/linear_model/_logistic.py:818:
ConvergenceWarning: lbfgs failed to converge (status=1):
STOP: TOTAL NO. of ITERATIONS REACHED LIMIT.
Increase the number of iterations (max_iter) or scale the data as shown in:
https://scikit-learn.org/stable/modules/preprocessing.html
Please also refer to the documentation for alternative solver options:
https://scikit-learn.org/stable/modules/linear\_model.html#logistic-regression
extra_warning_msg=_LOGISTIC_SOLVER_CONVERGENCE_MSG,
/usr/local/lib/python3.7/dist-packages/sklearn/linear_model/_logistic.py:818:
ConvergenceWarning: lbfgs failed to converge (status=1):
STOP: TOTAL NO. of ITERATIONS REACHED LIMIT.
Increase the number of iterations (max_iter) or scale the data as shown in:
https://scikit-learn.org/stable/modules/preprocessing.html
Please also refer to the documentation for alternative solver options:
https://scikit-learn.org/stable/modules/linear\_model.html#logistic-regression
extra_warning_msg=_LOGISTIC_SOLVER_CONVERGENCE_MSG,
/usr/local/lib/python3.7/dist-packages/sklearn/linear_model/_logistic.py:818:
ConvergenceWarning: lbfgs failed to converge (status=1):
STOP: TOTAL NO. of ITERATIONS REACHED LIMIT.
Increase the number of iterations (max_iter) or scale the data as shown in:
https://scikit-learn.org/stable/modules/preprocessing.html
Please also refer to the documentation for alternative solver options:
https://scikit-learn.org/stable/modules/linear\_model.html#logistic-regression
extra_warning_msg=_LOGISTIC_SOLVER_CONVERGENCE_MSG,
```

Output

0.823920227920228